TRACE

(Traçabilité, Réemploi et Affectation Circulaire des Équipements)

TRACE - DRFIP PARIS

FINANCES PUBLIQUES

Plan d'Architecture et d'Optimisation
- 1. Module : Gestion de l'Inventaire (Mobilier)
  - Consultation et Recherche Globale
  - Création d'Équipements (Unitaire ou par Lot)
  - Réaffectation Rapide par Scan (Douchette)
  - Mise à jour en Masse via Fichier (.txt)
  - Exportation CSV
- 2. Module : Catalogue National (Gabarits)
  - Application de la Charte de Nommage
  - Gestion du Référentiel (CRUD)
  - Constructeur Dynamique de Caractéristiques (Attributs JSON)
- 3. Module : Administration & Outillage
  - Gestion des Comptes et Référentiels (UA / Lieux)
  - Mise au Rebut Massive et Génération de PV
  - Générateur d'Étiquettes (QR Code)
- 4. Couche Transverse : Traçabilité & Infrastructure
  - Authentification et Sessions Sans État (Stateless)
  - Journal d'Audit Intégral (Audit Trail)
  - Hygiène des données (Tâches Automatisées)
- 5. Application Satellite : TRACE-SCAN (Inventaire Nomade)
  - Fonctionnement Hors-Ligne (Offline-first)
  - Scan Haute Performance et Ergonomie
  - Exportation et Interopérabilité
- 6. Déploiement et Administration Système (instrace.sh)
  - Provisionnement et Sécurité Serveur
  - Initialisation de la Base de Données et PostgREST

Proxy Inverse et Sécurité Web (Nginx)
Outillage d'Exploitation : Sauvegarde & Restauration
7. Maintenance et Code Applicatif (Développeurs)
7.1. Structure HTML, Imports et Dépendances (index.html)
7.2. Cartographie des fonctions JS (app.js)
7.3. Réflexion en cours : Évolution et découpage de l'architecture Front-end
7.4. Gestion des Grands Volumes et Conflits Simultanés (Back-end)
7.5. Architecture de l'Application Satellite (TRACE-SCAN)
8. Sécurité, Nomenclature (BOM) et Audit de Code
8.1. Nomenclature des Dépendances (BOM)
8.2. Politique de Veille et Mises à Jour (CVE)
8.3. Audit de Code et Protections Applicatives (XSS / JWT)
8.4. Sécurité de l'Infrastructure et des Données (Back-end)
9. Hypothèse de Mise en Exploitation (Modèle RACI)
Organisation des rôles
Choix de l'Infrastructure et Hébergement
Matrice RACI (Run & Maintenance)
Exigences opérationnelles et clauses de retrait
10. Observation : Démarche Open Source et Mutualisation
Mutualisation et Bien Commun

Propos préliminaires

La Direction Régionale des Finances Publiques (DRFIP) de Paris a exprimé le besoin de disposer d'un outil informatique moderne pour assurer le suivi précis de son parc mobilier. L'utilisation de QR Codes a été identifiée comme la solution la plus efficace pour automatiser les inventaires, faciliter les mouvements de matériel et assurer une traçabilité sans faille.

L'Établissement de Services Informatiques (ESI) de Paris a été sollicité pour la mise en œuvre de ce logiciel. Une phase de recherche a d'abord été menée au sein du Socle Interministériel du Logiciel Libre (SILL) pour identifier une solution existante mutualisable, mais celle-ci s'est avérée infructueuse. Par ailleurs, le Responsable de la Sécurité des Systèmes d'Information (RSSI) a interdit l'usage de solutions tierces commerciales sur le réseau informatique de la DGFIP. Cette décision est motivée par des impératifs de résilience informatique et la nécessité de maîtriser les risques de sécurité liés aux dépendances logicielles extérieures.

En réponse à ces besoins et contraintes, le logiciel TRACE a été conçu "sur-mesure". Il s'agit d'une application souveraine, développée en interne, garantissant une maîtrise totale du code source et une intégration parfaite aux infrastructures sécurisées de l'administration.Par ailleurs, le code source est rendu public et auditable, ce qui s'inscrit dans une démarche d'ouverture et de transparence (open source).

Plan d'Architecture et d'Optimisation

Ce document détaille l'architecture fonctionnelle de l'application TRACE (Traçabilité, Réemploi et Affectation Circulaire des Équipements), structurée par grands modules.

Objet de l'application :

L'application TRACE a pour vocation principale de moderniser et fluidifier la gestion du parc de mobilier (notamment au sein de la DGFIP). Elle permet aux agents de recenser les équipements, de suivre précisément leurs affectations physiques (services, bâtiments) et leurs statuts (en service, en maintenance, disponible pour réemploi). Son but ultime est d'encourager l'économie circulaire en offrant une vue centralisée et en temps réel du cycle de vie du mobilier, depuis son intégration au catalogue jusqu'à sa réaffectation ou sa mise au rebut définitive.

Pour chaque fonctionnalité, les mécanismes d'optimisation (côté client et serveur) sont mis en évidence pour garantir la performance sur de très grands volumes de données.

1. Module : Gestion de l'Inventaire (Mobilier)

C'est le cœur opérationnel de l'application, destiné aux agents pour le suivi au quotidien.

Consultation et Recherche Globale

- **Description :** Affichage sous forme de tableau interactif du parc de mobilier avec tris par colonnes et filtres combinés (Modèle, Service affecté, Statut). La barre de recherche est globale et "intelligente" : elle fouille simultanément dans de multiples informations de l'équipement (numéro d'identification, remarques de l'agent, nom du modèle, et même les caractéristiques techniques cachées).
- **Optimisation de l'Interface (Front-end) : Temporisation de la frappe**
  - *Pour tous :* L'application "patiente" une fraction de seconde (300 millisecondes) après votre dernière touche tapée avant de lancer la recherche au travers de l'inventaire. Cela évite de surcharger inutilement le système à chaque lettre tapée et maintient la fluidité de votre écran.
  - *Pour les techniciens :* Implémentation d'un *debouncing* sur le champ de recherche pour lisser les appels API réseau et préserver les performances du thread principal JavaScript.
- **Optimisation du Serveur et de la Base de données (Back-end) : Pagination et Indexation Vectorielle**
  - *Pour tous (Chargement progressif et rapidité) :* Au lieu d'afficher l'intégralité du catalogue d'un coup (ce qui ferait planter votre ordinateur), le système télécharge les équipements par "paquets" de 50. De plus, pour que la recherche textuelle soit instantanée malgré des centaines de milliers d'objets, le système utilise une méthode de découpage des mots similaire à un index de dictionnaire ultra-sophistiqué.
  - *Pour les techniciens :* 1. L'API exploite l'entête HTTP Range de PostgREST pour garantir une **pagination côté serveur** stricte, évitant le transfert et le rendu coûteux du DOM.
    2. Côté PostgreSQL, la recherche textuelle abandonne les requêtes séquentielles classiques (type ILIKE '%...%' qui scannent toute la table) pour s'appuyer sur un **index GIN Trigram** (extension pg_trgm, opérateur gin_trgm_ops). Ce mécanisme découpe le texte en blocs de trois caractères (trigrammes) et transforme la recherche en opérations mathématiques vectorielles, ce qui est extrêmement performant.

Création d'Équipements (Unitaire ou par Lot)

- **Description :** Formulaire d'ajout au parc. L'agent sélectionne un modèle, un lieu, une affectation et la quantité d'équipements à ajouter. L'identifiant métier unique (ex: MOB-000123) est généré automatiquement par le système.

- **Optimisation des performances : Création en masse et Sécurité**
  - *Pour tous (Rapidité et Fiabilité) :* Si vous devez créer 100 chaises identiques d'un coup, l'application ne fait pas 100 allers-retours avec le serveur. Elle envoie tout dans un seul "colis" pour que la création soit instantanée. De plus, un mécanisme de sécurité strict garantit que si deux agents ajoutent du matériel exactement à la même seconde, ils n'obtiendront jamais le même numéro d'identification.
  - *Pour les techniciens :* L'application effectue un **Bulk Insert** (insertion en masse) en envoyant un seul tableau JSON à l'API PostgREST, qui le traduit en une unique requête INSERT SQL. La gestion de l'auto-incrémentation de l'ID métier est totalement déportée au niveau de la base de données via un **Trigger PostgreSQL**. Cela prévient de manière infaillible les conflits d'accès concurrents (*race conditions*) lors de saisies simultanées.

Réaffectation Rapide par Scan (Douchette)

- **Description :** Interface optimisée pour le terrain. L'agent fige une destination (Service/Lieu) et scanne les codes-barres à la chaîne.
- **Optimisation des performances : Saisie ultra-rapide et sans interruption**
  - *Pour tous (Efficacité sur le terrain) :* L'écran est conçu pour que l'agent n'ait jamais besoin d'utiliser sa souris. Une fois la destination choisie, il lui suffit de "biper" chaque équipement avec sa douchette. Le système enregistre instantanément chaque meuble et affiche un petit message de confirmation qui ne bloque pas l'écran, permettant de scanner des dizaines d'équipements à la minute.
  - *Pour les techniciens :* L'interface force le *focus* automatique sur le champ de saisie (input) et intercepte l'événement clavier de la touche "Entrée" généré nativement par le lecteur de code-barres. Cela déclenche immédiatement une requête API PATCH unitaire et asynchrone vers PostgREST. Le feedback de réussite ou d'échec est ajouté dynamiquement au DOM (historique des scans) sans force le rechargement de la page ni bloquer le thread principal.

Mise à jour en Masse via Fichier (.txt)

- **Description :** Mise à jour de l'affectation ou du statut de dizaines d'équipements simultanément en chargeant un simple fichier texte. Typiquement, ce fichier est généré par l'application mobile satellite trace-scan utilisée par l'agent en mobilité sur le terrain.
- **Optimisation des performances : Traitement de masse sans attente**
  - *Pour tous (Zéro attente et fiabilité) :* Lorsque l'agent revient du terrain avec sa liste d'équipements scannés sur son téléphone ou sa douchette, il lui suffit de glisser-déposer le fichier de compte-rendu dans l'interface. L'application va instantanément lire ce fichier, éliminer automatiquement les erreurs (comme si l'agent avait bipé deux fois le même

meuble), et tout mettre à jour côté serveur en une fraction de seconde, le tout sans bloquer ni ralentir la page web.
    - *Pour les techniciens :* L'extraction des identifiants se fait directement en mémoire côté navigateur via l'API FileReader, éliminant le besoin d'uploader le fichier. Le code JavaScript nettoie la liste et utilise un objet Set pour garantir une déduplication parfaite (O(1)). Le front-end envoie ensuite une unique requête **Bulk Update** à l'API (en exploitant le paramètre PostgREST ?columns=id_metier). Le serveur traite l'ensemble des modifications au sein d'une seule transaction SQL, garantissant ainsi les propriétés ACID (si une mise à jour échoue, tout s'annule, protégeant l'intégrité de la base).

Exportation CSV

- **Description :** Téléchargement direct de la liste des équipements (en tenant compte des filtres appliqués à l'écran) sous forme de fichier tableur. Ce fichier est spécifiquement formaté pour s'ouvrir de manière parfaite sous Microsoft Excel, sans aucun problème d'accents ou de caractères bizarres.
- **Optimisation des performances : Génération locale sans surcharge**
    - *Pour tous (Immédiateté) :* Lorsque vous cliquez sur "Exporter", votre navigateur internet va lui-même fabriquer le fichier Excel directement sur votre ordinateur en une fraction de seconde. Cela évite de faire patienter le serveur central et vous garantit d'obtenir votre fichier instantanément, même s'il contient des dizaines de milliers de lignes.
    - *Pour les techniciens :* L'application récupère les données brutes via une requête API unique (en désactivant la pagination HTTP Range). Ensuite, tout le traitement se fait en mémoire côté client. Le fichier CSV est généré localement via l'API Blob, auquel on ajoute stratégiquement l'octet BOM UTF-8 (\ufeff) pour forcer Excel à lire le bon encodage. Le serveur est ainsi totalement exempté de toute charge de génération, de stockage ou de nettoyage de fichiers d'export temporaires.

2. Module : Catalogue National (Gabarits)

Ce référentiel permet de standardiser les modèles d'équipements à travers l'administration. Pour maintenir une base de données propre, claire et exploitable par tous, ce module s'appuie rigoureusement sur une **Charte de Nommage** obligatoire lors de la création des gabarits.

Application de la Charte de Nommage

- **Description :** Standardisation de la façon dont les modèles sont nommés, codifiés et détaillés, afin d'éviter les doublons de saisie (ex: éviter d'avoir "Chaise bleue", "Siège de bureau azur" et "Fauteuil" pour le même objet exact).
- **Optimisation de la qualité des données :**
  - *Pour tous (Lisibilité et simplicité) :* Grâce à cette charte, chaque modèle reçoit une référence courte et logique en 3 lettres et 3 chiffres (ex: BUR-001 pour le premier bureau, ASS-042 pour une assise). Les noms descriptifs vont toujours à l'essentiel ("Armoire haute à rideaux"). Les détails variables (comme la couleur ou les dimensions) ne polluent plus le titre, mais sont soigneusement rangés dans des cases à part, ce qui rend les menus déroulants et les recherches limpides.
  - *Pour les techniciens :* La charte impose une gouvernance et une structuration stricte des données : format de la référence imposé (Regex ^[A-Z]{3}-\d{3}$), normalisation des attributs JSON en *snake_case* sans accents (ex: largeur_cm, reference_marche), et typage rigoureux (utilisation stricte de booléens true/false, de vrais null plutôt que des chaînes vides, et de valeurs numériques brutes sans unités textuelles). Cette rigueur est fondamentale pour que l'indexation GIN de PostgreSQL soit performante et permette des requêtes analytiques croisées fiables.

Gestion du Référentiel (CRUD)

- **Description :** Création, modification et suppression des modèles d'équipements classés par grandes catégories (Bureau, Assise, Rangement, etc.).
- **Optimisation de la saisie : Auto-numérotation intelligente**
  - *Pour tous (Gain de temps et zéro erreur) :* Lors de la création d'un nouveau modèle d'armoire ou de chaise, l'agent n'a plus besoin de chercher dans tout le catalogue quel était le dernier numéro utilisé. Dès qu'il choisit la catégorie, l'application scanne toute seule la liste et propose automatiquement la référence suivante disponible (par exemple, si "BUR-003" existe déjà, elle inscrira "BUR-004"). Cela accélère grandement le travail et empêche la création de doublons.
  - *Pour les techniciens :* L'application front-end (en JavaScript) écoute l'événement de sélection de la catégorie, déduit le préfixe correspondant selon la Charte (BUR, ASS, RAN...), puis filtre le tableau global des gabarits conservé en mémoire

(state.gabarits) pour calculer le prochain incrément mathématique (MAX(id) + 1). Le champ de saisie est pré-rempli dynamiquement, offrant un gain d'UX immédiat sans nécessiter de requête serveur supplémentaire coûteuse.

Constructeur Dynamique de Caractéristiques (Attributs JSON)

- **Description :** Interface permettant d'ajouter un nombre illimité de caractéristiques spécifiques à un modèle (ex: "couleur", "longueur_cm", "année_acquisition", "ref_marche"). Au lieu d'avoir un formulaire figé, l'agent peut créer autant de nouvelles cases (couples "Clé/Valeur") qu'il le souhaite pour décrire un objet avec précision, sans jamais avoir besoin de demander à un informaticien de modifier le code de l'application.
- **Optimisation de la structure : Flexibilité et Recherche instantanée**
  - *Pour tous (Sur-mesure et rapidité) :* Habituellement, ajouter un nouveau type d'information (comme le fait qu'un bureau soit "motorisé") nécessite de lourdes modifications informatiques. Ici, le système range ces détails comme de petites étiquettes intelligentes attachées au modèle d'équipement. L'astuce majeure de l'outil, c'est que la base de données est capable de lire à l'intérieur de toutes ces étiquettes à la vitesse de l'éclair, même pour trouver une "couleur chêne" noyée parmi des milliers d'équipements.
  - *Pour les techniciens :* L'application exploite le type de données **JSONB** (JSON Binaire compressé) de PostgreSQL. Au lieu de figer le schéma SQL avec des dizaines de colonnes souvent vides, ou d'utiliser un modèle "Entité-Attribut-Valeur" (EAV) trop lourd, les caractéristiques sont stockées dans une colonne JSON structurée. L'optimisation majeure repose sur la création d'un **index GIN natif** (idx_gabarits_caract) sur ce champ. Cela permet d'exécuter des requêtes de recherche textuelle ou structurelle directement à l'intérieur des nœuds profonds du JSON de manière quasi-instantanée, sans provoquer de "Full Table Scan".

3. Module : Administration & Outillage

Espace réservé aux profils administrateurs pour la gouvernance de l'application.

Gestion des Comptes et Référentiels (UA / Lieux)

- **Description :** Interfaces de création des comptes agents/admins, réinitialisation des mots de passe, et maintien des listes de services d'affectation (SAGES) et de lieux physiques.
- **Optimisation de l'accès : Sécurité et Ségrégation**
  - *Pour tous (Sécurité garantie) :* Le système sépare strictement ce qu'un agent peut faire de ce qu'un administrateur peut modifier. De plus, les mots de passe ne sont jamais stockés en clair : ils sont transformés en une empreinte secrète indéchiffrable par un algorithme très puissant, garantissant la sécurité des accès même en cas de problème.
  - *Pour les techniciens :* **Ségrégation des droits (RBAC)** stricte déléguée à PostgreSQL (Rôles agent vs administrateur). Le hachage des mots de passe (Bcrypt via pgcrypto) est traité directement par le moteur de base de données (fonction creer_utilisateur), ce qui évite de manipuler des mots de passe en clair dans la couche serveur ou applicative.

Mise au Rebut Massive et Génération de PV

- **Description :** Suppression physique de lots d'équipements en fin de vie à partir d'un fichier texte, avec génération automatique d'un Procès-Verbal PDF (formaté selon le DSFR) attestant de la sortie du parc.
- **Optimisation de traitement : Découpage et Génération locale**
  - *Pour tous (Stabilité et fluidité) :* Si vous devez mettre au rebut 5 000 chaises, l'application ne va pas essayer de tout supprimer en un seul gros bloc (ce qui ferait planter le système). Elle va les supprimer par petits paquets invisibles de 100. De plus, le document officiel PDF est généré directement par votre navigateur, ce qui vous permet de l'obtenir immédiatement.
  - *Pour les techniciens :* **Traitement par lots (Chunking)**. Les identifiants sont découpés par tranches de 100 pour la recherche et la suppression afin de ne pas engorger le réseau ni faire planter le serveur sur des milliers de lignes. Le PDF est généré 100% côté client via jsPDF, libérant le backend de cette charge de calcul (CPU-intensive).

Générateur d'Étiquettes (QR Code)

- **Description :** Outil complet pour générer des planches A4 d'étiquettes QR prêtes à l'impression, avec personnalisation du design (Marianne, couleurs, corrections d'erreurs).

- **Optimisation de l'impression : Rendu natif au millimètre**
  - *Pour tous (Qualité d'impression) :* L'outil de création de QR codes n'utilise pas de simples images qui pourraient devenir floues, mais dessine les étiquettes mathématiquement. Cela garantit une impression nette et un alignement parfait au millimètre près sur vos planches d'étiquettes autocollantes.
  - *Pour les techniciens :* Rendu client natif (Canvas via qrcode-styling.js) couplé à une grille CSS native (display: grid) parfaitement calibrée pour l'impression (dimensions en millimètres). Cela élimine les requêtes serveur pour générer des images et garantit une qualité vectorielle lors de l'impression.

4. Couche Transverse : Traçabilité & Infrastructure

Authentification et Sessions Sans État (Stateless)

- **Description :** Connexion de l'utilisateur et maintien de sa session.
- **Optimisation de la connexion : Laissez-passer numérique**
  - *Pour tous (Rapidité) :* Plutôt que de vérifier dans un registre à chaque fois que vous cliquez sur un bouton, l'application vous donne un "laissez-passer numérique" crypté lors de votre connexion. Ce laissez-passer permet au système de fonctionner de manière ultra-fluide, même s'il y a des milliers d'agents connectés simultanément.
  - *Pour les techniciens :* Utilisation de **Jetons JWT**. L'API (PostgREST) ne stocke aucune session en mémoire. La validité est vérifiée localement en JavaScript (date d'expiration) et validée mathématiquement par PostgreSQL à chaque requête, rendant l'application hautement scalable et légère.

Journal d'Audit Intégral (Audit Trail)

- **Description :** Enregistrement automatique de toute action (Création, Modification, Suppression) sur le mobilier, avec l'horodatage, l'agent responsable et la nature précise de la modification.
- **Optimisation de la traçabilité : Boîte noire inaltérable**
  - *Pour tous (Fiabilité absolue) :* Telle la boîte noire d'un avion, le système enregistre de manière automatique et invisible absolument toutes les actions réalisées sur un équipement (qui, quoi, quand). Il est techniquement impossible d'effacer ou de contourner cette historisation, ce qui garantit une traçabilité parfaite.
  - *Pour les techniciens :* Déport de la logique métier via un **Trigger PostgreSQL** (trig_audit_mobiliers). Le journal est généré au plus bas niveau, garantissant que même une modification en base de données contournant l'application web (ex: via une requête SQL directe) sera tracée de manière inaltérable.

Hygiène des données (Tâches Automatisées)

- **Description :** Maintien de l'espace disque et de la sécurité.
- **Optimisation de la maintenance : Robots de nettoyage**
  - *Pour tous (Sérénité) :* Le système s'entretient tout seul la nuit. Des robots nettoient automatiquement les vieilles traces d'historique inutiles pour ne pas saturer l'espace, et réalisent des sauvegardes complètes pour s'assurer qu'aucune donnée ne soit jamais perdue.
  - *Pour les techniciens :* Script de déploiement (instrace.sh) configurant des **tâches Cron natives** sous Linux pour la

sauvegarde tournante de la base (pg_dump) et la purge nocturne des logs d'audit vieux de plus de 3 mois.

5. Application Satellite : TRACE-SCAN (Inventaire Nomade)

Application mobile compagnon spécialement conçue pour accompagner les agents en mobilité sur le terrain, leur permettant de scanner des équipements à la volée.

Fonctionnement Hors-Ligne (Offline-first)

- **Description :** Capacité à utiliser l'application de scan même dans des endroits sans aucune couverture réseau (caves, sous-sols, archives).
- **Optimisation :**
  - *Pour tous (Disponibilité) :* L'application s'installe directement sur le smartphone de l'agent. Une fois ouverte une première fois, elle n'a plus jamais besoin d'internet pour fonctionner. L'agent peut descendre au sous-sol, scanner tout son matériel, et remonter en sécurité.
  - *Pour les techniciens :* TRACE-SCAN est une **Progressive Web App (PWA)**. Un script d'arrière-plan appelé **Service Worker** (sw.js) intercepte toutes les requêtes réseau et sert les ressources essentielles (fichiers HTML, CSS et librairie JS) directement depuis le cache du téléphone. Aucune base de données locale complexe n'est requise, tout se passe en mémoire vive.

Scan Haute Performance et Ergonomie

- **Description :** Utilisation de l'appareil photo du smartphone pour lire rapidement les QR Codes sans nécessiter de matériel spécifique.
- **Optimisation :**
  - *Pour tous (Confort visuel et haptique) :* Le scanner est fluide et réactif. Si on laisse la caméra trop longtemps sur la même étiquette, l'application bloque intelligemment le "double-scan" accidentel. De plus, le téléphone émet une légère vibration à chaque réussite, évitant à l'agent de devoir regarder son écran en permanence.
  - *Pour les techniciens :* Utilisation de l'API navigateur pour le flux vidéo (html5-qrcode.min.js), couplée à un algorithme qui garde en mémoire le dernier scan pour bloquer les doublons immédiats. Le feedback combine l'API matérielle navigator.vibrate et des transitions CSS non-bloquantes (ajout temporaire d'une classe scan-success) pour optimiser le rendu graphique sans faire chuter les FPS.

Exportation et Interopérabilité

- **Description :** Création d'un rapport de scan en fin de tournée pour réintégration dans le système central TRACE.
- **Optimisation :**
  - *Pour tous (Facilité de transfert) :* À la fin de sa tournée, l'agent clique sur "Terminer la session". L'application crée instantanément un petit fichier qu'il peut envoyer par mail, par messagerie, ou conserver sur son téléphone. Ce fichier est nativement reconnu par le module de "Mise à jour en Masse" de TRACE.
  - *Pour les techniciens :* Le fichier d'export est généré localement sous forme de Blob. Il est préfixé par un **BOM UTF-8** (\ufeff) garantissant qu'il sera encodé et lu correctement par Excel ou le bloc-notes, évitant tout problème d'encodage de caractères. L'application utilise nativement l'API navigator.share sur mobile pour ouvrir la fenêtre de partage du système (Mail, Drive, etc.), avec un mécanisme de secours par création de lien (URL.createObjectURL) si le partage natif échoue.

6. Déploiement et Administration Système (instrace.sh)

Espace à destination exclusive des administrateurs système (SysAdmin) et équipes d'infrastructure. Ce module décrit les opérations réalisées par le script d'automatisation instrace.sh lors d'un déploiement initial sur un serveur Debian 13 vierge.

Provisionnement et Sécurité Serveur

- **Description :** Installation des fondations (moteur de base de données, serveur web frontal) et sécurisation cryptographique des échanges réseau.
- **Optimisation de l'installation : Déploiement "Clé en main"**
  - *Pour tous (Mise en route rapide) :* Le script agit comme un maître de chantier autonome. Il transforme un serveur Linux complètement vide en un serveur TRACE prêt à l'emploi. Il installe tous les "moteurs" nécessaires et s'occupe de créer lui-même le certificat de sécurité (le cadenas HTTPS) pour garantir que les communications entre le navigateur de l'agent et le serveur soient privées.
  - *Pour les techniciens :* Installation non interactive via apt des dépendances requises (postgresql-17, nginx, openssl). Le script génère automatiquement des certificats SSL auto-signés valables 1 an via openssl req dans le dossier /etc/ssl/trace. Ces certificats se voient attribuer des droits restrictifs (chown postgres:postgres et chmod 600 sur la clé privée), garantissant que seuls les processus essentiels (Nginx root worker et PostgreSQL) puissent y accéder.

Initialisation de la Base de Données et PostgREST

- **Description :** Création intégrale du schéma de la base de données relationnelle, initialisation du moteur API PostgREST et importation automatique des données de l'ancien système (fichiers CSV).
- **Optimisation de l'architecture : L'intelligence dans la donnée**
  - *Pour tous (Robustesse) :* Plutôt que de confier l'intelligence de l'application à un programme intermédiaire souvent fragile, le script configure la base de données pour qu'elle soit elle-même le cerveau de TRACE. Il crée les "armoires" pour le mobilier, invente les comptes administrateurs par défaut, et intègre tout seul les fichiers Excel historiques. Il configure également le petit moteur (PostgREST) qui fera le lien direct entre la page web et la base.
  - *Pour les techniciens :* Déploiement complet en "Database-Centric". Le script exécute un DDL PostgreSQL complet : suppression/création de la base parc_mobilier_dgfip, injection des Types (ENUM), Tables, Séquences, Vues SQL optimisées, et configuration stricte du RBAC interne (divagil, agent, administrateur). L'import CSV (\copy) de la reprise de données est automatisé, suivi du re-calcul des séquences et du lancement de la commande ANALYZE. Le binaire PostgREST est installé

et géré via un fichier service systemd (trace-api.service) qui garantit son redémarrage automatique en cas de crash (Restart=always).

Proxy Inverse et Sécurité Web (Nginx)

- **Description :** Configuration du serveur web frontal (nginx) pour orienter les flux réseau, sécuriser l'API et héberger les fichiers statiques (HTML/JS/CSS).
- **Optimisation des flux : Le vigile d'entrée**
  - *Pour tous (Protection) :* Le système place Nginx en tant que "vigile" à l'entrée du serveur. Si vous tapez l'adresse sans sécurisation, il vous force à utiliser la version sécurisée (HTTPS). Il sert vos pages web très rapidement, et protège la base de données en s'assurant que les requêtes ne passent que par une "petite porte" très surveillée.
  - *Pour les techniciens :* Configuration d'un bloc serveur *Reverse Proxy* en HTTP/2. Une redirection permanente 301 force le trafic du port 80 vers le port 443. Le site frontend est servi directement via try_files depuis /var/www/html/. Pour le backend, la route /api/ est proxyfiée vers l'instance locale de PostgREST sur le port 3000. Des optimisations de sécurité sont intégrées : flag HttpOnly; Secure sur les cookies, transfert de l'IP réelle de l'utilisateur (X-Real-IP), et masquage actif des signatures logicielles via server_tokens off et proxy_hide_header X-Powered-By.

Outillage d'Exploitation : Sauvegarde & Restauration

- **Description :** Mise en place des routines autonomes de survie de la donnée et des scripts de reprise d'activité interactive (PRA).
- **Optimisation de l'administration : "Time Machine" et Externalisation (NFS)**
  - *Pour tous (Tranquillité d'esprit) :* Vous n'avez pas besoin de penser aux sauvegardes. Le serveur est configuré pour faire un "cliché" compressé complet de tous vos mobiliers et historiques chaque nuit à 2h00 du matin. Pour éviter de perdre les données si le serveur physique venait à casser, le dossier où sont rangées les sauvegardes est conçu pour être facilement connecté à un disque dur distant ou un coffre-fort numérique de l'administration. Si un jour une erreur catastrophique survient, un petit outil permet à l'informaticien de restaurer la version de la veille facilement.
  - *Pour les techniciens :* Mise en place stricte de tâches via les dossiers /etc/cron.d/ (plus pérenne que le crontab utilisateur). Le script trace_backup.sh exécute un pg_dump -Fc (format custom, ultra compressé) quotidien à 02h00 vers /mnt/savetrace, avec une rotation autonome supprimant les archives vieilles de plus de 30 jours via find -mtime. **Préconisation d'externalisation :** Il est fortement recommandé à l'administrateur système de configurer un point de montage réseau (NFS ou CIFS/SMB) ciblant le répertoire /mnt/savetrace via le fichier /etc/fstab. Cela garantit que le dump SQL écrit par la tâche cron est physiquement déporté sur une baie de stockage tierce. Enfin, l'outil trace_restore.sh est

créé : un script Bash interactif paginé qui parse les dumps locaux et encapsule la commande complexe pg_restore --clean pour faciliter le travail des équipes de Niveau 1 en cas de sinistre.

7. Maintenance et Code Applicatif (Développeurs)

Espace à destination exclusive des développeurs en charge de la maintenance, du débogage ou de l'évolution du code de l'application TRACE.

7.1. Structure HTML, Imports et Dépendances (index.html)

Le fichier index.html est le point d'entrée unique de l'application web (Single Page Application - SPA).

- **Architecture SPA :** Le routage n'utilise pas de rechargement de page complet. Les différentes "pages" (Inventaire, Catalogue, Administration) sont définies comme des div masquées ou affichées dynamiquement via le basculement des classes CSS .active (manipulées par la fonction showSubView()).
- **Dépendances UI / UX (DSFR) :** L'interface repose intégralement sur le Système de Design de l'État (DSFR v1.14.3). Les imports incluent les feuilles de styles core (dsfr.min.css) et utilitaires (utility.main.min.css), ainsi que les scripts JS nécessaires au fonctionnement des composants natifs (modales, menus, alertes).
- **Dépendances Logiques (Utilitaires tiers) :**
  Pour ne pas dépendre d'un serveur applicatif (Node.js/Python), la génération du Procès-Verbal de mise au rebut se fait côté client. Le fichier importe statiquement jspdf.umd.min.js (coeur PDF) et jspdf.plugin.autotable.min.js (génération de tableaux) depuis le dossier local ./utils/.
- **Fichiers Locaux :** styles.css (surcharges cosmétiques et flexbox) et app.js (contenant toute la logique métier).

7.2. Cartographie des fonctions JS (app.js)

Le fichier app.js orchestre actuellement l'ensemble de la logique applicative. Les fonctions y sont regroupées par périmètre fonctionnel :

- **État Global (State Management) :**
  L'objet let state = {...} est la source de vérité. Il stocke les données chargées (tableaux d'objets), les maps pour des résolutions d'ID rapides (O(1)), les critères de recherche et les paramètres de pagination pour toutes les vues.
- **Authentification & Sécurité :**
  - authentifier(e) : Call API vers le RPC PostgreSQL login, stockage du token dans le sessionStorage.
  - parseJwt(token) : Décrypte le payload pour vérifier la date d'expiration (exp) et extraire le rôle (agent / administrateur).
  - getHeaders() : Prépare le Bearer Token et l'en-tête Prefer requis par PostgREST.

    - escapeHTML(str) : **Fonction critique** de neutralisation des inputs (mitigation des failles XSS) lors du rendu dans le DOM.
- **Gestion du Mobilier (Inventaire principal) :**
    - loadData() : Fetch asynchrone des référentiels (Gabarits, Structures, Lieux) en Promise.all().
    - applyFiltersAndSort() : Prépare les paramètres d'URL (filtres, recherche textuelle via opérateur ilike, entête Range) et interge la vue SQL vue_mobiliers_recherche.
    - renderMobilierPage() : Génère et injecte le HTML des lignes du tableau à partir de state.filteredData.
    - saveMobilier(e, mode) : Gère à la fois le POST (Bulk Insert pour la création par lot) et le PATCH (édition unitaire).
    - processScan(event) & processFileImport() : Fonctions dédiées à la saisie rapide (lecture du DOM via listener "Enter" pour la douchette, et API FileReader / objet Set pour la déduplication et l'import de masse de fichiers texte).
- **Gestion des Gabarits (Catalogue) :**
    - suggestNextReference() : Calcule automatiquement l'incrément (ex: BUR-004) en parsant les références existantes pour garantir le respect de la charte de nommage.
    - saveGabarit(e) : Compile les attributs dynamiques saisis via l'interface du constructeur et les convertit en un objet pur pour l'insertion PostgreSQL (JSONB).
- **Administration & Exports :**
    - generateRebutPDF(data) : Utilise jsPDF et l'objet Canvas pour générer un document PDF certifié (avec logo Marianne) à partir de la sélection d'équipements à supprimer.
    - exportToCSV() : Désactive temporairement la pagination pour récupérer toutes les données correspondantes aux filtres, puis utilise l'API Blob pour télécharger un fichier structuré (avec BOM \ufeff pour Excel).

7.3. Réflexion en cours : Évolution et découpage de l'architecture Front-end

Le fichier app.js centralise actuellement toute la logique de l'application (~800 lignes). Bien que cette architecture monolithique soit simple à déployer (pas de build step, pas de Webpack/Vite), elle devient complexe à maintenir et limite le travail collaboratif entre plusieurs développeurs.

Cible d'architecture préconisée : Les modules ECMAScript (ES Modules)

Pour préparer les évolutions futures de TRACE, une refonte du app.js vers une architecture modulaire native (utilisant les mots-clés import et export supportés par tous les navigateurs modernes) est à l'étude. Ce découpage se structurerait idéalement ainsi :

1. main.js : Point d'entrée, initialisation (DOM Content Loaded) et routage principal.
2. api/fetchClient.js : Centralisation des requêtes réseau, de la gestion du JWT (getHeaders()), et interception globale des erreurs

(ex: 401 Unauthorized pour déconnexion automatique).

3. store/state.js : Isoler l'objet state et créer des "getters/setters" pour éviter les mutations directes et non contrôlées des variables globales.
4. modules/mobilier.js : CRUD de l'inventaire et algorithmes de pagination/recherche textuelle.
5. modules/gabarits.js : Logique de la charte de nommage et constructeur dynamique du JSON.
6. utils/domHelper.js : Fonctions transverses de manipulation du DOM (escapeHTML, showAlert, fillSelect).
7. utils/exportHelper.js : Isoler la lourde logique de génération PDF (jsPDF) et d'export CSV.

Impact sur l'intégration :

Ce découpage nécessitera de modifier la balise d'inclusion dans le fichier index.html : <script type="module" src="js/main.js"></script>. L'avantage est qu'elle n'implique **aucune dépendance côté serveur** ni aucun outil de compilation (Node.js reste inutile), respectant ainsi la philosophie ultra-légère du projet TRACE.

7.4. Gestion des Grands Volumes et Conflits Simultanés (Back-end)

- **Description :** Mécanismes de protection et d'optimisation (côté base de données et API) pour garantir l'intégrité des données et la fluidité des réponses lors d'une utilisation intensive par plusieurs centaines d'agents simultanément.
- **Optimisation des gros volumes : Traitement à la source**
  - *Pour tous (Fluidité garantie) :* L'application est conçue pour ne jamais s'essouffler, même avec des centaines de milliers d'équipements enregistrés. Si vous modifiez 1 000 chaises d'un coup, l'interface envoie un seul "gros colis" au serveur au lieu de faire 1 000 petits allers-retours. Le travail lourd est calculé directement dans la salle des machines, sans saturer votre connexion internet ni figer votre écran.
  - *Pour les techniciens :* L'architecture "Database-Centric" (via PostgREST) élimine le goulot d'étranglement typique des ORM (comme Sequelize ou Prisma) tournant sur un serveur Node.js/Python intermédiaire. Les opérations ensemblistes (*Bulk Inserts* ou *Bulk Updates*) transmettent un payload JSON directement interprété en **une unique transaction SQL**. Couplé à la pagination stricte par entêtes HTTP (Range) et à l'indexation vectorielle (GIN/Trigram), la consommation mémoire (RAM) serveur reste constante. Les requêtes s'exécutent en quelques millisecondes indépendamment du volume de la table ciblée.
- **Gestion des conflits d'accès lors de la création d'équipements (Mobilier)**
  - *Pour tous (Zéro doublon) :* Imaginez que deux agents, situés dans deux villes différentes, cliquent sur "Ajouter" exactement au même millième de seconde pour créer du nouveau matériel. L'application dispose d'un "feu de circulation"

mathématique infaillible. Il force l'une des demandes à patienter une fraction de seconde de manière invisible, garantissant que les deux équipements recevront des numéros de série (ID métier) uniques. Il est impossible d'écraser la donnée de son collègue.
    - *Pour les techniciens :* La prévention des accès concurrents n'est pas gérée maladroitement en JavaScript. Elle est totalement déléguée au moteur ACID de PostgreSQL. La génération automatique des identifiants (ex: MOB-XXXXXX) s'appuie sur des **séquences SQL transactionnelles** et des Triggers BEFORE INSERT. De même, lors de mises à jour simultanées, le moteur applique un verrouillage granulaire au niveau de la ligne modifiée (**Row-Level Locking**). Cela évite nativement toute anomalie de type "Lost Update" (mise à jour perdue) sans avoir besoin d'écrire de logique de blocage complexe (*mutex*) au niveau du serveur web.
- **Point de vigilance prioritaire (Dette technique) : Faille de concurrence sur les Référentiels (Gabarits, Services, Lieux)**
    - *Pour tous (Le risque de collision) :* Autant la création de milliers d'équipements est blindée, autant la création des "catégories de l'application" (comme le numéro d'un nouveau modèle de bureau ou l'ajout d'un nouveau service administratif) possède une petite faiblesse temporaire. Si deux administrateurs cliquent sur "Créer un modèle" exactement à la même seconde, le système web croira que le numéro "BUR-004" est libre pour les deux. L'un des deux administrateurs recevra un message d'erreur.
    - *Pour les techniciens :* Contrairement aux équipements (qui utilisent un Trigger sécurisé côté serveur), l'auto-incrémentation des Gabarits (suggestNextReference()) et la création de nouveaux Lieux ou Services (UA) sont actuellement calculées **côté client (Front-end)** en JavaScript via un simple algorithme MAX(id) + 1. Cela crée une vulnérabilité classique d'accès concurrent de type **TOCTOU** (*Time-Of-Check to Time-Of-Use*). En cas d'enregistrement rigoureusement simultané par deux administrateurs, PostgREST rejettera la seconde insertion en renvoyant une erreur de violation de contrainte d'unicité (UNIQUE constraint violation - HTTP 409 Conflict).
    - *Action de remédiation préconisée pour les développeurs :* Il faut impérativement déporter cette logique de numérotation directement dans PostgreSQL. La solution consistera à remplacer le calcul JS par des séquences SQL dédiées (gabarits_bur_seq, gabarits_ass_seq) associées à un Trigger BEFORE INSERT sur la table des référentiels, garantissant ainsi une atomicité totale de l'opération, indépendamment du client.

7.5. Architecture de l'Application Satellite (TRACE-SCAN)

L'outil nomade TRACE-SCAN est une **Progressive Web App (PWA)** indépendante. Elle ne communique pas directement avec l'API PostgREST afin de garantir un fonctionnement 100% hors-ligne. Voici la description des fichiers la composant :

- **index.html (Cœur applicatif et UI) :** Fichier central contenant la structure HTML (adaptée pour mobile, empêchant le

défilement global) et la logique JavaScript.
  - *Logique de scan :* Initialise la caméra via la classe Html5Qrcode, bloque les doublons consécutifs (scans accidentels), et gère le retour haptique (navigator.vibrate) et visuel (ajout dynamique de la classe .scan-success).
  - *Logique d'export :* Transforme le tableau des scans en un fichier texte. Intègre un préfixe BOM UTF-8 (\ufeff) critique pour garantir la lecture correcte de l'encodage sous Microsoft Excel. Tente d'abord d'utiliser l'API Web Share (navigator.share) avec un mécanisme de *fallback* classique basé sur URL.createObjectURL en cas d'échec ou d'environnement non compatible.
- **sw.js (Service Worker) :** Le pilier de la stratégie "Offline-First". Ce script s'installe en arrière-plan et intercepte les requêtes réseau (évènement fetch). Il met en cache (trace-scan-cache-v1.1) l'ensemble des ressources statiques lors de la phase d'install, permettant à l'agent d'utiliser l'application même dans un sous-sol sans couverture réseau.
- **manifest.json :** Fichier descriptif JSON standardisant le comportement de la PWA. Il dicte au système d'exploitation mobile les paramètres d'installation sur l'écran d'accueil (icônes SVG/PNG, nom court, et mode d'affichage standalone pour masquer l'URL du navigateur).
- **Dépendances embarquées :**
  - html5-qrcode.min.js : La librairie open-source chargée du décodage du flux vidéo en temps réel. **Important :** Cette librairie est volontairement hébergée localement (et non chargée via un CDN externe) pour pouvoir être pré-mise en cache par le Service Worker.
  - *Ressources DSFR* : Le header officiel (Marianne) et quelques éléments CSS sont intégrés pour assurer la cohérence visuelle avec la plateforme web principale.

8. Sécurité, Nomenclature (BOM) et Audit de Code

Espace dédié aux experts en cybersécurité, RSSI et mainteneurs techniques. Ce module détaille la gestion des dépendances logicielles (Bill of Materials) et les mécanismes de protection contre les vulnérabilités informatiques mis en place dans TRACE.

8.1. Nomenclature des Dépendances (BOM)

- **Description :** Inventaire strict des composants externes utilisés pour fabriquer l'application, afin d'identifier rapidement les éléments à mettre à jour en cas de découverte d'une faille de sécurité mondiale.
- **La liste des "ingrédients" informatiques :**
  - *Pour tous (Transparence) :* TRACE n'est pas une boîte noire. Elle est construite avec des éléments logiciels reconnus et très surveillés (comme le socle "Debian" ou le design officiel de l'État "DSFR"). Tenir cette liste à jour nous permet de savoir exactement quel composant réparer si une alerte de sécurité est diffusée sur internet.
  - *Pour les techniciens :* L'application repose sur une stack très réduite, minimisant drastiquement la surface d'attaque (pas de NodeJS, pas de NPM, pas de Python) :
    - **OS & Infra :** Debian 13 (Trixie), Nginx (via dépôts officiels), OpenSSL (via dépôts officiels).
    - **Base de données & API :** PostgreSQL v17 (dépôts Debian), PostgREST (Binaire pré-compilé téléchargé depuis GitHub).
    - **Interface (Front-end) :** Vanilla JS/HTML5 natifs, DSFR v1.14.3 (hébergé localement).
    - **Librairies JS tierces (via CDN) :** jsPDF v2.5.1, jsPDF-AutoTable v3.5.25, QR Code Styling v1.6.0-rc.1.

8.2. Politique de Veille et Mises à Jour (CVE)

- **Description :** Processus permettant de s'assurer que TRACE reste étanche face aux nouvelles techniques de piratage découvertes au fil du temps.
- **Anticipation des menaces :**
  - *Pour tous (Maintenance continue) :* L'ordinateur central (le serveur) se met à jour presque tout seul pour les fondations. En revanche, pour les "pièces détachées" spécifiques (comme le générateur de PDF ou l'outil qui fait le lien avec la base de données), l'équipe informatique est abonnée à des alertes et doit les remplacer manuellement dès qu'une faiblesse est signalée.
  - *Pour les techniciens :* Le suivi de sécurité (CVE) est hybride. Les composants système (Debian, Nginx, PostgreSQL, OpenSSL) sont maintenus par des apt-get update && apt-get upgrade planifiés mensuellement. En revanche, **PostgREST** et les **librairies Front-end (DSFR, jsPDF)** exigent une veille manuelle active sur leurs dépôts GitHub respectifs. En cas de

vulnérabilité, il est de la responsabilité du mainteneur de télécharger le nouveau binaire de PostgREST ou de mettre à jour la version des liens CDN dans le fichier index.html.

8.3. Audit de Code et Protections Applicatives (XSS / JWT)

- **Description :** Mesures intégrées directement dans le code source de la page web pour empêcher le vol de session ou l'injection de scripts malveillants.
- **La désinfection des saisies utilisateurs :**
  - *Pour tous (Protection du navigateur) :* Si un agent malveillant ou inattentif tape du "code de piratage" au lieu d'écrire "Bureau en bois" dans la case des remarques, l'application est programmée pour nettoyer ce texte. Elle le rendra totalement inoffensif avant de l'afficher sur l'écran des autres collègues. De plus, elle n'utilise pas de mot de passe persistant pour mémoriser votre connexion, mais un "badge temporaire" volatil.
  - *Pour les techniciens :* * **Mitigation XSS (Cross-Site Scripting) :** TRACE n'utilisant pas de framework réactif (comme React ou Angular qui auto-échappent les chaînes), le fichier app.js repose systématiquement sur une fonction utilitaire critique : escapeHTML(str). Cette fonction passe les inputs dans un élément DOM inerte (div.textContent) avant de les réinjecter, neutralisant ainsi toute balise <script> ou événement onerror malicieux.
    - **Sécurisation de Session (JWT) :** L'authentification est *stateless* (sans état serveur). Le jeton JSON Web Token est stocké dans le sessionStorage du navigateur. Ce choix, couplé à une courte durée de validité vérifiée systématiquement (côté JS et côté API), empêche les attaques par fixation de session.

8.4. Sécurité de l'Infrastructure et des Données (Back-end)

- **Description :** Boucliers cryptographiques et cloisonnement strict des accès directement au niveau de la salle des machines (la base de données).
- **Cloisonnement et Chiffrement fort :**
  - *Pour tous (Mots de passe illisibles) :* Même l'administrateur informatique en chef du système est incapable de lire votre mot de passe. Ils sont transformés en un charabia mathématique impossible à décrypter. Par ailleurs, toutes les données qui voyagent entre votre ordinateur et le serveur transitent dans un "tuyau blindé" (le petit cadenas HTTPS de votre navigateur).
  - *Pour les techniciens :* * **Mots de passe et Hachage :** Les mots de passe ne quittent jamais le backend en clair. Ils sont traités nativement par PostgreSQL via l'extension pgcrypto et la fonction crypt(), utilisant l'algorithme Blowfish (Bcrypt) avec génération de sel cryptographique aléatoire.
    - **Contrôle d'Accès (RBAC) :** Le principe du moindre privilège est appliqué. L'API web n'a pas les droits de super-administrateur. Elle utilise un rôle limité (divagil) qui n'a le droit de "prendre l'identité" (via GRANT) que des rôles

fonctionnels agent ou administrateur selon ce qui est dicté par le payload du JWT signé.

- **Chiffrement en transit (TLS) :** Nginx joue le rôle de terminateur SSL. Il impose des communications chiffrées (port 443 obligatoire, HTTPS), s'appuyant sur les certificats générés par openssl lors de l'installation, protégeant ainsi l'application des attaques de type "Man-in-the-Middle" (MitM).

9. Hypothèse de Mise en Exploitation (Modèle RACI)

Ce module définit l'organisation du travail et la répartition des responsabilités une fois l'application TRACE déployée et utilisée au quotidien par les agents.

Organisation des rôles

- **Description :** Pour que l'application fonctionne parfaitement sur le long terme, il est indispensable de savoir "Qui fait quoi". Le modèle **RACI** répond à cette question.
- **Définition pour tous (Le casting du projet) :**
  - **MOA (Maîtrise d'Ouvrage) :** Le "Client" ou "Métier" (ex: La DGFIP). C'est lui qui exprime le besoin et valide que l'application rend bien le service attendu.
  - **MOE (Maîtrise d'Œuvre) :** Le chef d'orchestre technique. Il s'assure que le projet est réalisable et coordonne les équipes informatiques.
  - **Développement :** L'équipe de programmeurs qui écrit, modifie et corrige le code de l'application.
  - **Exploitation (SysAdmin) :** Les mécaniciens du serveur. Ils gèrent la machine physique, installent l'application et s'assurent qu'elle ne tombe pas en panne.
  - **Hébergement :** Le fournisseur d'accès qui prête la machine physique ou le cloud, ainsi que le réseau internet.
  - **Assistance :** Le centre d'aide (Helpdesk) que les agents peuvent appeler s'ils n'arrivent pas à cliquer sur un bouton.
- **Définition pour les techniciens (La matrice RACI) :**
  Le tableau ci-dessous ventile les responsabilités opérationnelles selon la norme RACI :
  - **(R) Réalisateur (Responsible) :** Celui qui exécute concrètement la tâche.
  - **(A) Approbateur (Accountable) :** L'autorité finale, celui qui rend des comptes. (Un seul "A" possible par ligne).
  - **(C) Consulté (Consulted) :** Celui dont l'avis est requis avant ou pendant l'action.
  - **(I) Informed (Informed) :** Celui qui est tenu au courant une fois l'action terminée.

Choix de l'Infrastructure et Hébergement

- **Description :** Arbitrage et sélection de la plateforme accueillant le serveur TRACE.
- **Optimisation de la souveraineté et de l'écoresponsabilité : Cloud vs On-Premise**
  - *Pour tous (Les enjeux de l'hébergement) :* Nous devons choisir entre installer l'application sur des serveurs physiques au sein des bureaux de la DRFIP ou utiliser le "nuage" (Cloud) de l'État. Le Cloud est privilégié car il est plus écologique (les

serveurs sont partagés et optimisés pour moins consommer) et conforme aux règles nationales qui demandent de centraliser l'informatique pour plus de sécurité et de simplicité.

- *Pour les techniciens :* L'arbitrage entre un hébergement sur serveurs physiques locaux (on-premise DRFIP) et un hébergement sur **Cloud Souverain (Nubo, Pi)** doit intégrer trois dimensions critiques :
  1. **Doctrine "Cloud au Centre" :** La circulaire du Premier Ministre impose l'usage du cloud pour tout nouveau projet numérique de l'État afin d'accélérer la transformation et garantir la résilience.
  2. **Écoresponsabilité et Sobriété Numérique :** Les centres de données mutualisés (Nubo/Pi) présentent un indicateur d'efficacité énergétique (PUE) bien meilleur que les salles serveurs locales. La virtualisation permet un "right-sizing" des ressources, évitant le gaspillage de matériel physique sous-exploité.
  3. **MCO et Résilience :** L'hébergement sur Cloud Souverain est une condition *sine qua non* pour le passage en production. Il permet de déléguer la gestion de la couche matérielle et offre nativement des services de haute disponibilité et de redondance géographique compatibles avec les exigences du PRA (Plan de Reprise d'Activité).

Matrice RACI (Run & Maintenance)

Activité / Tâche opérationnelle	Service en charge	MOA	MOE	Développement	Exploitation	Hébergement	Assistance
Définition des besoins & Évolutions	DRFIP Paris	**A / R**	C	C	I	I	C
Fourniture de l'infrastructure (Serveurs, Réseau)	À définir	I	I	I	C	**A / R**	I
Déploiement logiciel initial (instrace.sh)	À définir	I	**A**	C	**R**	I	I

Maintien en Condition Opérationnelle (MCO) & Sauvegardes	À définir	I	**A**	I	**R**	C	I
Correction d'anomalies (MCO Applicatif / Bugs)	À définir	I	**A**	**R**	C	I	I
Support Utilisateur (Niveau 1 - Questions d'usage)	À définir	**A**	C	I	I	I	**R**
Support Technique (Niveau 2/3 - Pannes complexes)	À définir	I	**A**	**R**	**R**	C	C

Exigences opérationnelles et clauses de retrait

- **Description :** Analyse critique de la viabilité du projet TRACE au-delà de son développement technique.
- **Optimisation de la pérennité : Sécuriser les moyens avant de lancer**
  - *Pour tous (La survie du projet) :* Créer une application est une chose, la faire vivre en est une autre. Un projet informatique est comme une voiture : même si elle est neuve et fonctionne parfaitement, on ne peut pas la mettre sur la route si on n'a personne pour l'entretenir, payer le garage et porter assistance en cas de panne. Si l'administration n'est pas capable de désigner les équipes qui s'occuperont de TRACE après son lancement, le projet devra être arrêté pour ne pas gaspiller plus d'énergie. Le code restera toutefois disponible en tant que bien commun numérique, permettant ainsi de capitaliser sur le travail accompli.
  - *Pour les techniciens (Gouvernance et Exploitabilité) :* Tout projet informatique présente un risque opérationnel majeur. Une application, même fonctionnellement parfaite, ne peut être mise en production si les moyens d'exploitabilité ne sont pas garantis.
    - **Critère de "No-Go" :** En cas d'incapacité à assigner des services responsables dans les cases "À définir" de la matrice RACI ci-dessus, le projet TRACE sera considéré comme orphelin. Par mesure de sobriété numérique et de sécurité, il devra être abandonné avant tout passage en production.
    - **Pérennité Open-source :** Dans ce cas de figure, le code source sera archivé et maintenu sur GitHub sous licence libre. Cette démarche permet de conserver la valeur métier produite et garantit la réutilisabilité du patrimoine logiciel sans repartir de zéro, dans une logique de mutualisation et de transparence.

10. Observation : Démarche Open Source et Mutualisation

Le code source de l'application TRACE est disponible publiquement sur un dépôt GitHub et publié sous licence Creative Commons. Cette initiative l'inscrit pleinement dans la grande famille des logiciels libres et Open Source.

Mutualisation et Bien Commun

- **Description :** L'ouverture du code source permet de partager l'outil avec l'ensemble de la sphère publique, évitant ainsi le financement multiple de développements identiques par les deniers publics.
- **Bénéfices de la démarche :**
  - *Pour tous (Un bien commun et gratuit) :* L'application ayant été financée par des fonds publics (la DGFIP), son "plan de fabrication" est rendu public. Ainsi, toute structure souhaitant moderniser la gestion de son propre mobilier peut récupérer gratuitement TRACE, l'installer et l'utiliser. C'est le principe de la réutilisation appliqué aux logiciels informatiques.
  - *Pour les techniciens (Public Money, Public Code) :* Le code est hébergé sur un dépôt GitHub accessible publiquement sous licence Creative Commons. Cette ouverture permet non seulement un audit de sécurité transparent et continu par la communauté, mais ouvre également la voie à une proposition d'intégration au **SILL (Socle Interministériel du Logiciel Libre)** administré par la DINUM. Intégrer le catalogue officiel du SILL certifie la qualité institutionnelle du logiciel, facilite son référencement technique au sein de l'État, et officialise son statut de patrimoine numérique mutualisable.